

```

mysql> /* TCL and Constraints
/*> Transaction: the process of combining all related operations into a single
unit and executing them according to the rule("either all or none")
/*> example: funds transfer
/*> two operations: debit from sender's account, and credit from receiver's
account should be successful or fail together
/*> if one succeeds and other fails, it leads to data inconsistency problems.
/*>
/*> There are two types of transactions:
/*> 1. local transactions
/*> 2. global transactions
/*>
/*> 1. local transactions: here, all operations of a transaction are executed on
the same database.
/*> eg: funds transfer from one account to another account where both accounts
belong to the same bank.
/*>
/*> 2. global transactions: here, all operations of a transaction are executed
on different databases.
/*> eg: funds transfer from one account to another account where the accounts
belong to different banks.
/*>
/*> ACID properties
/*> A - Atomicity: all or nothing
/*> C - Consistency: The database remains in a valid state
/*> I - Isolation: transactions do not interface with each other
/*> D - Durability: committed changes are permanent
/*>
/*> Transaction Control Language commands:
/*> 1. commit
/*> 2. rollback
/*> 3. save point
/*>
/*> commit: it is used to permanently save all changes made during the current
transaction into the database.
/*> once committed: changes become permanent, other users can see the changes,
you cannot undo the changes using rollback.
/*>
/*> rollback: it is used to undo all changes made in the current transaction
before a commit.
/*> if something goes wrong, rollback restores the database to its previous
state.
/*> */
mysql> drop table students;
Query OK, 0 rows affected (0.11 sec)

mysql> create table students(
-> std_in int primary key,
-> std_name varchar(50),
-> marks int

```

```
-> );  
Query OK, 0 rows affected (0.12 sec)
```

```
mysql> start transaction;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> insert into students values(111,'malli',97),(222,'mahi',100);  
Query OK, 2 rows affected (0.05 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> commit;  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 111 | malli | 97 |  
| 222 | mahi | 100 |  
+-----+-----+-----+  
2 rows in set (0.04 sec)
```

```
mysql> start transaction;  
Query OK, 0 rows affected (0.00 sec)
```

```
Query OK, 1 row affected (0.03 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 111 | malli | 60 |  
| 222 | mahi | 100 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

```
mysql> rollback;  
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 111 | malli | 97 |  
| 222 | mahi | 100 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

```
mysql> start transaction;
```

Query OK, 0 rows affected (0.00 sec)

mysql> delete from students where std_in = 111;

Query OK, 1 row affected (0.05 sec)

mysql> commit;

Query OK, 0 rows affected (0.04 sec)

mysql> rollback;

Query OK, 0 rows affected (0.00 sec)

mysql> select * from students;

std_in	std_name	marks
222	mahi	100

1 row in set (0.00 sec)

mysql> start transaction;

Query OK, 0 rows affected (0.00 sec)

mysql> insert into students values(333,'sunny',60),(444,'bunny',86);

Query OK, 2 rows affected (0.00 sec)

Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from students;

std_in	std_name	marks
222	mahi	100
333	sunny	60
444	bunny	86

3 rows in set (0.00 sec)

mysql> update students set std_name = 'mani' where std_in = 222;

Query OK, 1 row affected (0.00 sec)

Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from students;

std_in	std_name	marks
222	mani	100
333	sunny	60
444	bunny	86

3 rows in set (0.00 sec)

```
mysql> delete from students where std_in = 333;  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 222 | mani | 100 |  
| 444 | bunny | 86 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

```
mysql> rollback;  
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 222 | mahi | 100 |  
+-----+-----+-----+  
1 row in set (0.00 sec)
```

```
mysql> start transaction;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> insert into students values(333,'sunny',60),(444,'bunny',86);  
Query OK, 2 rows affected (0.00 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 222 | mahi | 100 |  
| 333 | sunny | 60 |  
| 444 | bunny | 86 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)
```

```
mysql> update students set std_name = 'mani' where std_in = 222;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> select * from students;  
+-----+-----+-----+  
| std_in | std_name | marks |  
+-----+-----+-----+  
| 222 | mani | 100 |  
| 333 | sunny | 60 |
```

444	bunny	86
-----	-------	----

3 rows in set (0.00 sec)

```
mysql> savepoint sp1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> delete from students where std_in = 444;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from students;
+-----+-----+-----+
| std_in | std_name | marks |
+-----+-----+-----+
| 222   | mani    | 100   |
| 333   | sunny   | 60    |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> rollback to sp1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select * from students;
+-----+-----+-----+
| std_in | std_name | marks |
+-----+-----+-----+
| 222   | mani    | 100   |
| 333   | sunny   | 60    |
| 444   | bunny   | 86    |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> /* Stored Procedures
```

```
/*> It is a named and precompiled collection of one or more SQL statements that
is stored inside the database and can be executed, called whenever required instead
of sending long block of SQL from an application every time, the application simply
calls the procedure by name.
```

```
/*>
```

```
/*> Why use stored procedures?
```

```
/*> 1. performance
```

```
/*> 2. reusability
```

```
/*> 3. security
```

```
/*> 4. reduced network traffic
```

```
/*> 5. maintainability
```

```
/*> 6. consistency
```

```
/*>
```

```
/*> advantages
```

```
/*> 1. faster execution after completion
```

```
/*> 2. reduced duplicate sql
```

```
/*> 3. supports transaction, loops, conditions, variables, ... etc
```

```

/*> 4. easy maintenance
/*>
/*> disadvantages
/*> 1. debugging can be difficult
/*> 2. vendor specific syntax
/*> 3. complex procedures may become hard to maintain.
/*>
/*> syntax:
/*> DELIMITER //
/*> create procedure procedure_name()
/*> begin
/*> --- sql statements
/*> end //
/*> DELIMITER;
/*>
/*> IN paramater: it is used to pass a value into a stored procedure.
/*> the procedure can use the value, but changes made to the parameter inside
the procedure are not returned to the caller.
/*>
/*> syntax:
/*> create procedure procedure_name(IN parameter_name datatype)
/*> begin
/*> sql statements
/*> end ;
/*> delimiter ;
/*> */
mysql> drop table employees;
Query OK, 0 rows affected (0.14 sec)

mysql> CREATE DATABASE companydb;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE companydb;
Database changed
mysql>
mysql> CREATE TABLE employees (
->     emp_id INT PRIMARY KEY,
->     emp_name VARCHAR(50),
->     department VARCHAR(50),
->     salary DECIMAL(10,2),
->     experience INT,
->     city VARCHAR(50),
->     age INT
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> INSERT INTO employees
-> VALUES

```

```

-> (101, 'Ravi', 'IT', 45000, 2, 'Hyderabad', 24),
-> (102, 'Kiran', 'HR', 40000, 3, 'Vijayawada', 27),
-> (103, 'Suresh', 'IT', 55000, 5, 'Hyderabad', 30),
-> (104, 'Anil', 'Sales', 35000, 1, 'Chennai', 23),
-> (105, 'Priya', 'Finance', 60000, 6, 'Bangalore', 32),
-> (106, 'Sneha', 'IT', 50000, 4, 'Vizag', 28),
-> (107, 'Rahul', 'HR', 42000, 2, 'Hyderabad', 25),
-> (108, 'Arjun', 'Sales', 38000, 3, 'Chennai', 26),
-> (109, 'Divya', 'Finance', 65000, 7, 'Bangalore', 34),
-> (110, 'Manoj', 'IT', 70000, 8, 'Hyderabad', 35);

```

Query OK, 10 rows affected (0.07 sec)

Records: 10 Duplicates: 0 Warnings: 0

```
mysql> select * from employees;
```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24
102	Kiran	HR	40000.00	3	Vijayawada	27
103	Suresh	IT	55000.00	5	Hyderabad	30
104	Anil	Sales	35000.00	1	Chennai	23
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
107	Rahul	HR	42000.00	2	Hyderabad	25
108	Arjun	Sales	38000.00	3	Chennai	26
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

10 rows in set (0.04 sec)

```
mysql> delimiter //
```

```
mysql> create procedure getbyid(in eid int)
```

```
    -> begin
```

```
        -> select * from employees where emp_id = eid;
```

```
    -> end //
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> delimiter ;
```

```
mysql> call getbyid(101);
```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24

1 row in set (0.06 sec)

Query OK, 0 rows affected (0.06 sec)

```
mysql> call getbyid(110);
```

emp_id	emp_name	department	salary	experience	city	age
--------	----------	------------	--------	------------	------	-----

emp_id	emp_name	department	salary	experience	city	age
110	Manoj	IT	70000.00	8	Hyderabad	35

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> delimiter //

mysql> create procedure getEmployeeBySalary(in eid int)

-> begin

-> select salary from employees where emp_id = eid;

-> end //

Query OK, 0 rows affected (0.06 sec)

mysql> delimiter ;

mysql> call getEmployeeBySalary(101);

salary
45000.00

1 row in set (0.00 sec)

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> call getEmployeeBySalary(110);

salary
70000.00

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> call getEmployeeBySalary(105);

salary
60000.00

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> show procedure status where db = database();

Procedure	Database	Created	Modified	Comment
getEmployeeBySalary	database()			

Db	Name	Type	Definer	Modified
Created	Security_type	Comment	character_set_client	
collation_connection	Database Collation			
companydb	getbyid	PROCEDURE	root@localhost	2026-09-23 14:33:56
2026-09-23 14:33:56	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			
companydb	getEmployeeBySalary	PROCEDURE	root@localhost	2026-09-23 14:37:58
2026-09-23 14:37:58	DEFINER		cp850	
cp850_general_ci	utf8mb4_0900_ai_ci			

2 rows in set (0.06 sec)

```
mysql> delimiter //
mysql> create procedure getEmployeesByDepartment(in dpt varchar(50))
-> begin
-> select * from employees where department = dpt;
-> end //
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> delimiter ;
mysql> call getEmployeesByDepartment('IT');
```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24
103	Suresh	IT	55000.00	5	Hyderabad	30
106	Sneha	IT	50000.00	4	Vizag	28
110	Manoj	IT	70000.00	8	Hyderabad	35

4 rows in set (0.05 sec)

Query OK, 0 rows affected (0.07 sec)

```
mysql> call getEmployeesByDepartment('Sales');
```

emp_id	emp_name	department	salary	experience	city	age
104	Anil	Sales	35000.00	1	Chennai	23
108	Arjun	Sales	38000.00	3	Chennai	26

2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```
mysql> delimiter //
```

```
mysql> create procedure getEmployeesByRange(in min decimal(10,2), in max decimal(10,2))
```

```
-> begin
```

```
-> select * from employees where salary between min and max;
```

```
-> end //
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> delimiter ;
```

```
mysql> call getEmployeesByRange(40000,80000);
```

emp_id	emp_name	department	salary	experience	city	age
101	Ravi	IT	45000.00	2	Hyderabad	24
102	Kiran	HR	40000.00	3	Vijayawada	27
103	Suresh	IT	55000.00	5	Hyderabad	30
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
107	Rahul	HR	42000.00	2	Hyderabad	25
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

```
8 rows in set (0.06 sec)
```

```
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> call getEmployeesByRange(50000,80000);
```

emp_id	emp_name	department	salary	experience	city	age
103	Suresh	IT	55000.00	5	Hyderabad	30
105	Priya	Finance	60000.00	6	Bangalore	32
106	Sneha	IT	50000.00	4	Vizag	28
109	Divya	Finance	65000.00	7	Bangalore	34
110	Manoj	IT	70000.00	8	Hyderabad	35

```
5 rows in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> delimiter //
```

```
mysql> create procedure departmentExperience(in dept varchar(50))
```

```
-> begin
```

```
-> select * from employees where experience > 5 and department = dept;
```

```
-> end //
```

```
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> delimiter ;
```

```
mysql> call departmentExperience('IT');
```

emp_id	emp_name	department	salary	experience	city	age
103	Suresh	IT	55000.00	5	Hyderabad	30
106	Sneha	IT	50000.00	4	Vizag	28
110	Manoj	IT	70000.00	8	Hyderabad	35

```

+-----+-----+-----+-----+-----+-----+
| 110 | Manoj | IT | 70000.00 | 8 | Hyderabad | 35 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

Query OK, 0 rows affected (0.01 sec)

```
mysql> show procedure status where db = database();
```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| Db      | Name                                     | Type      | Definer      | Modified
| Created | Security_type | Comment | character_set_client |
collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| companydb | departmentExperience | PROCEDURE | root@localhost | 2026-09-23
14:57:13 | 2026-09-23 14:57:13 | DEFINER | | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getbyid | PROCEDURE | root@localhost | 2026-09-23
14:33:56 | 2026-09-23 14:33:56 | DEFINER | | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getEmployeeBySalary | PROCEDURE | root@localhost | 2026-09-23
14:37:58 | 2026-09-23 14:37:58 | DEFINER | | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getEmployeesByDepartment | PROCEDURE | root@localhost | 2026-09-23
14:45:28 | 2026-09-23 14:45:28 | DEFINER | | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
| companydb | getEmployeesByRange | PROCEDURE | root@localhost | 2026-09-23
14:52:16 | 2026-09-23 14:52:16 | DEFINER | | cp850
cp850_general_ci | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

```
mysql> /* OUT parameter
```

```
/*> it is used to return a value from the stored procedure to the caller.
```

```
/*> the caller provides a variable to receive the result.
```

```
/*>
```

```
/*> syntax:
```

```
/*> delimiter //
```

```
/*> create procedure procedure_name(OUT parameter_name datatype)
```

```
/*> begin
```

```
/*> set parameter_name = value;
```

```
/*> end //
```

```
/*> delimiter ;
```

```
/*> */
```

```
mysql>
```

```
mysql> -- return the total number of employees
mysql> delimiter //
mysql> create procedure totalEmployees(out total int)
-> begin
-> select count(*) into total from employees;
-> end //
```

Query OK, 0 rows affected (0.04 sec)

```
mysql> delimiter ;
mysql> set @total = 0;
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> call totalEmployees(@total);
Query OK, 1 row affected (0.03 sec)
```

```
mysql> select @total;
+-----+
| @total |
+-----+
|      10 |
+-----+
1 row in set (0.03 sec)
```

```
mysql> delimiter //
mysql> create procedure maxSalary(out maxsal decimal(10,2))
-> begin
-> select max(salary) into maxsal from employees;
-> end //
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> delimiter ;
mysql> set @highest = 0;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call maxSalary(@highest);
Query OK, 1 row affected (0.05 sec)
```

```
mysql> select @highest;
+-----+
| @highest |
+-----+
| 70000.00 |
+-----+
1 row in set (0.06 sec)
```

```
mysql> delimiter //
mysql> create procedure itCount(out count int)
-> begin
-> select count(emp_id) into count from employees where department = 'IT';
-> end //
```

Query OK, 0 rows affected (0.08 sec)

```
mysql> delimiter ;
```

```
mysql> set @it = 0;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> call itCount(@it);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> select @it as count;
```

count
4

1 row in set (0.00 sec)

```
mysql> /* IN-OUT parameter
```

```
/*> it is used when we want to send a value into the procedure and receive the  
modified value back
```

```
/*> INOUT = IN + OUT
```

```
/*>
```

```
/*> syntax:
```

```
/*> create procedure procedure_name(INOUT parameter_name datatype)
```

```
/*> */
```

```
mysql>
```

```
mysql> -- add 5000.0 to the specified emp_id
```

```
mysql> delimiter //
```

```
mysql> create procedure increaseSalary(in eno int, inout sal decimal(10,2))
```

```
-> begin
```

```
-> select salary+sal into sal from employees where emp_id = eno;
```

```
-> end //
```

Query OK, 0 rows affected (0.07 sec)

```
mysql> delimiter ;
```

```
mysql> set @newsalary = 0;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> set @new = 5000;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> call increasesalary(101,@new);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> select @new as new_salary;
```

new_salary
50000.00

1 row in set (0.00 sec)

```
mysql> -- increase salary by 10%
```

```
mysql> delimiter //
```

```
mysql> create procedure increaseten(in eno int, inout salary decimal(10,2))
```

```
-> begin
```

```
-> select salary + (salary*0.10) into salary from employees where emp_id =  
eno;
```

```
-> end //
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> delimiter ;
```

```
mysql> set @amount = 0;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> call increaseten(101,@amount);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> select @amount;
```

```
+-----+
```

```
| @amount |
```

```
+-----+
```

```
| 0.00 |
```

```
+-----+
```

1 row in set (0.00 sec)

```
mysql> drop procedure increaseten;
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> delimiter //
```

```
mysql> create procedure increaseten(in eno int, inout sal decimal(10,2))
```

```
-> begin
```

```
-> select salary + (salary*0.10) into sal from employees where emp_id = eno;
```

```
-> end //
```

Query OK, 0 rows affected (0.07 sec)

```
mysql> delimiter ;
```

```
mysql> set @amount = 0;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> call increaseten(101,@amount);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> select @amount;
```

```
+-----+
```

```
| @amount |
```

```
+-----+
```

```
| 49500.00 |
```

```
+-----+
```

1 row in set (0.00 sec)

```
mysql> set @amount = 0;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call increaseten(105,@amount);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @amount;
```

@amount

66000.00

```
1 row in set (0.00 sec)
```

```
mysql> notee
```